Experiment 8

Query Processing on Large Datasets Using Apache Hive

Subject: Data Science Lab | Environment: Hadoop 3.4.1 + Hive 4.2.1 | Mapped CO(s): CO2

1. Aim

To perform SQL-like querying, aggregation, and analysis of large datasets using Apache Hive.

2. Key Concepts

Apache Hive

Open-source data warehouse on Hadoop. Users query HDFS data with HiveQL, and Hive turns queries into MapReduce/Tez/Spark jobs.

HiveQL

SQL-like query language of Hive (SELECT, WHERE, GROUP BY, ORDER BY, JOIN, ...).

Metastore

Database that stores table definitions (schema, location, SerDe). This lab uses an embedded Derby metastore.

Schema-on-read

Data stays as files in HDFS; the schema is applied only when the data is read, not when it is loaded.

Managed vs External table

Managed: Hive owns the data under /user/hive/warehouse and DROP deletes it. External: data stays at its own location; DROP removes only metadata.

SerDe

Serializer/Deserializer that tells Hive how to read rows. OpenCSVSerde handles quoted CSV fields like "Braund, Mr. Owen Harris".

CTAS

CREATE TABLE ... AS SELECT. Creates a new table from a query result (used here to build a typed ORC table).

ORC

Optimized Row Columnar file format: compressed, columnar, fast for analytical queries.

3. Hive Architecture

Component	Role
Beeline / CLI	Client that submits HiveQL
HiveServer2	JDBC/ODBC service that accepts queries
Driver	Compiler + optimizer + executor of queries
Metastore	Table schemas and HDFS locations
Execution engine	MapReduce (this lab), Tez, or Spark
HDFS	Stores the table data files

4. HiveQL Statements Used

Statement	Purpose
CREATE DATABASE titanic_db	Create a database
CREATE TABLE ... ROW FORMAT SERDE	Define a table and how rows are parsed
LOAD DATA INPATH '...' INTO TABLE	Move an HDFS file into a table
CREATE TABLE t STORED AS ORC AS SELECT	Build a typed table from a query
SHOW DATABASES / SHOW TABLES	List databases / tables
DESCRIBE titanic	Show table schema
SELECT ... WHERE	Filter rows
GROUP BY + COUNT/AVG/SUM	Aggregate per group
ORDER BY ... DESC	Sort results
MAX() / MIN()	Largest / smallest value

5. Algorithm

- 1. Start HDFS (NameNode, DataNode) and upload Titanic-Dataset.csv to /user/lab8/titanic.
- 2. Configure hive-site.xml (Derby metastore, warehouse dir, execution engine = mr).
- 3. Initialise the metastore: schematool -dbType derby -initSchema.
- 4. CREATE DATABASE titanic_db and a staging table using OpenCSVSerde.
- 5. LOAD DATA INPATH the CSV into the staging table.
- 6. CTAS into a typed ORC table titanic (INT/DOUBLE columns, empty strings -> NULL).
- 7. Run SELECT, WHERE, GROUP BY, ORDER BY and aggregate queries with beeline.
- 8. Analyse and verify the results.

6. Aggregation Formulas

$$AVG(fare) = \frac{1}{n}\sum_{i=1}^{n} fare_i$$

$$Survival\ \% = \frac{SUM(survived)}{COUNT(*)} \times 100 = AVG(survived) \times 100$$

AVG and other aggregate functions ignore NULL values (e.g. the 177 passengers without an age).

7. Results (Titanic, 891 passengers)

Pclass	Passengers	Survived	Survival %	Avg Age	Avg Fare
1	216	136	62.96	38.23	84.15
2	184	87	47.28	29.88	20.66
3	491	119	24.24	25.14	13.68

By sex: female 314 passengers, 74.20% survived; male 577 passengers, 18.89% survived.

8. Exercise Answers

Exercise 1: Display only female passengers

SELECT passengerid, name, pclass, age, fare, survived FROM titanic WHERE sex = 'female'; -> 314 rows.

Exercise 2: Average fare for each passenger class

SELECT pclass, ROUND(AVG(fare), 2) FROM titanic GROUP BY pclass; -> 1: 84.15, 2: 20.66, 3: 13.68.

Exercise 3: Number of passengers who survived

SELECT COUNT(*) FROM titanic WHERE survived = 1; -> 342 (38.38%).

Exercise 4: Sort passengers by age (descending)

SELECT ... FROM titanic WHERE age IS NOT NULL ORDER BY age DESC; -> 714 rows. Oldest: Barkworth, Mr. Algernon Henry Wilson (80). Youngest: 0.42 years.

Exercise 5: Maximum and minimum fare

SELECT MAX(fare), MIN(fare) FROM titanic; -> max 512.33 (3 passengers), min 0.0 (15 passengers).

9. Hive vs Pig vs RDBMS

	Hive	**Pig**	**RDBMS**
Language	HiveQL (declarative)	Pig Latin (procedural)	SQL
Best for	Reporting, BI	ETL pipelines	Transactions
Schema	On read	Optional	On write
Latency	High (batch)	High (batch)	Low
Data size	Petabytes	Petabytes	GB-TB

10. Viva Q&A

Q: What is Apache Hive?

A: A data warehouse system on Hadoop that lets you query HDFS data using SQL-like HiveQL.

Q: What is the Hive Metastore?

A: A database storing table metadata: column names, types, SerDe, and HDFS location.

Q: What is schema-on-read?

A: Hive applies the schema when reading data, so loading is just a file move.

Q: Managed vs external table?

A: DROP on a managed table deletes data; on an external table only the metadata is removed.

Q: What does LOAD DATA INPATH do?

A: Moves the HDFS file into the table's warehouse directory (no parsing).

Q: Why OpenCSVSerde?

A: Names contain commas inside quotes; the default delimiter would split them.

Q: Why was a WHERE clause used to drop the header?

A: The MapReduce CTAS job did not apply skip.header.line.count, so the header row was filtered explicitly.

Q: How does Hive execute a query?

A: Driver parses and optimizes HiveQL, then runs it as MapReduce/Tez jobs.

Q: Why are simple SELECT ... LIMIT queries fast?

A: Hive uses a fetch task (reads files directly) without launching a MapReduce job.

Q: ORDER BY vs SORT BY?

A: ORDER BY gives a total order using one reducer; SORT BY sorts within each reducer only.

Q: What is partitioning?

A: Splitting a table into sub-directories by a column value (e.g. year) to skip unneeded data.

Q: What is bucketing?

A: Hashing rows into a fixed number of files by a column, useful for sampling and joins.

Q: How do aggregates treat NULL?

A: COUNT(col), AVG, SUM, MAX, MIN ignore NULLs; COUNT(*) counts all rows.

Q: Is Hive suitable for OLTP?

A: No. It is designed for batch analytics with high latency, not row-level transactions.

11. One-Line Definitions

Hive

SQL-on-Hadoop data warehouse

HiveQL

Hive's SQL-like query language

Metastore

Catalog of table metadata

Beeline

JDBC command-line client for Hive

HiveServer2

Service that runs client queries

SerDe

Reads/writes rows in a file format

Warehouse

/user/hive/warehouse - default table location

CTAS

Create a table from a SELECT result

Partition

Sub-directory per column value

Bucket

Hash-based file split of a table